

Relazione Progetto Sistemi Mobile

<https://github.com/StatJon/ProgettoSistemiMobile>

Descrizione e scopo del progetto

Il progetto consiste nella creazione di un videogioco a turni vecchio stile, chiamato "Dungeons & Dungeons".

Il gioco è composto da due schermate principali gestite da relativi viewmodel (MainMenu e GameScreen) a cui si aggiungono due schermate semplici senza viewmodel (LoadingScreen ed EndScreen).

Il Menu principale permette di avviare una nuova partita con uno dei personaggi sbloccati o continuare una eventuale partita lasciata in sospeso.

Il gioco consiste nello svolgimento in sequenza di una serie di combattimenti tra il personaggio del giocatore e un nemico (recuperato tramite API, vedere dopo) fino al termine degli incontri previsti.

É prevista la persistenza dei dati del giocatore tramite Room per salvare il numero di vittorie totali (su cui si basa la lunghezza della partita e gli sblocchi dei personaggi) e dei dati della sessione di gioco attiva, in modo da poter uscire e riprendere la partita in un secondo momento.

Il salvataggio è automatico ed effettuato a ogni fine round, uscire dal gioco e premere continua nel menù principale permette di riprendere dalla fine dell'ultimo scontro effettuato.

Sono state utilizzate chiamate API tramite Retrofit assieme a Moshi verso <https://www.dnd5eapi.co/> per il fetching degli avversari e delle abilità utilizzate dal giocatore, inoltre sono stati usati mapper ad-hoc come convertitori tra i dati grezzi recuperati e salvati in Dto e le classi di dominio del gioco.

Struttura

Applicazione divisa in quattro moduli principali: app, domain, data, uicompose

App contiene gli endpoint della applicazione:

- GameApp per la inizializzazione delle dipendenze necessarie
- MainActivity per la inizializzazione della Ui. MainActivity richiama ScreenManager, una classe adibita alla gestione delle varie schermate.

Domain contiene tutte le classi di dominio e gli usecases, insieme racchiudono la business logic del app. Inoltre sono presenti le interfacce di repository (le cui implementazioni sono presenti in data) e le classi di dependency injection.

Data contiene i tre metodi di recupero dati utilizzati dalla app:

- Gamedata, un object locale contenente dati utili al gioco
 - Remote, contenente la logica per l'interfacciamento verso la API remota tramite Retrofit per le chiamate e Moshi per il recupero dei dati in forma di Dto.
 - Local contenente la logica e le chiamate verso il DB locale tramite Room
- A questi si aggiungono le implementazioni delle repository che utilizzano i metodi sopra per eseguire il recupero dei dati.
- Infine vi è l'implementazione dei RepositoryProvider.

UiCompose contiene il codice Android-Compose necessario per la creazione della UI. È l'unico modulo che contiene codice Android-Compose (assieme ad app\MainActivity.kt).

- ScreenManager è il contenitore principale dentro cui viene deciso quale Screen mostrare.
 - Vi sono due screen principali, MainMenu e GameScreen, a cui sono annessi i relativi ViewModel.
- EndScreen e LoadingScreen non possiedono ViewModel poichè sono screen semplici e senza logica.
- Tutti utilizzano components\common\UiConstants per evitare dati hardcoded e garantire mantenibilità.
- I due ViewModel utilizzano gli usecases per la logica di business e i StateFlow per mantenere il proprio stato aggiornato.
 - GameScreenViewModel contiene la logica necessaria per decidere cosa mostrare e orchestrare gli usecases.
 - GameScreen utilizza diversi @Composable tutti contenuti in components\gamescreen.
-

Punti di forza

- Indipendenza tra i moduli: Domain è completamente agnostico dagli altri moduli, Data è agnostico ad App e Uicompose. Ciò permette totale separazione tra la logica di business, il recupero dati e la visualizzazione e orchestrazione.
 - Uso estensivo di UseCases per tutti i casi di interazioni tra membri del Domain.
 - É stato rispettato rigorosamente il cablaggio tra Ui, ViewModel, ViewModelFactory, RepositoryProvider, UseCaseProvider, Repository(Interface + Implementation) e UseCases (Interface + Implementation).
 - Mapper tra API e domain: Usati mapper ad-hoc per Dto per permettere la maggiore compatibilità e coerenza possibile tra i dati recuperati dalla API e il Domain di gioco.
 - Logica di gioco a macchina ha stati con init+loop pattern: Simula un gameloop ma senza usare un update tick, utilizzando i cambi di stati per attendere e continuare la logica e ciclare il loop di gioco affiancato dal pattern init + loop per la corretta inizializzazione e svolgimento dei loop.
 - Channel per integrazione nei loop: Usato kotlinx.coroutines.Channel per permettere all'input utente di essere utilizzato all'interno del loop ed evitare di dover chiudere e reiniziare il loop a ogni azione dell'utente.
-

Possibili migliorie

- Implementazione di più feature di gioco secondarie e game design più avanzato.
 - Maggiore contenuto di base e più mapper ad-hoc per permettere una maggiore integrazione con spell e monsters reperibili dalla API dnd5e.
 - Polish generale UI/UX.
 - Animazioni degli Sprite (movimento degli sprite).
-

Assets pubblici e tools esterni utilizzati

<https://www.aseprite.org/>

<https://kenney.nl/assets>

<https://aekashics.itch.io/aekashics-librarium-librarium-static-batch-megapack>